



# UNIMAP

Navegación universitaria para estudiantes  
y visitantes en un campus nuevo

## CONTEXTO: Una escena demasiado conocida

### 1. Llegas al Campus

Primera clase, evento o tramite en un lugar que todavía no conoces.

### 2. Identificas el Bloque

No sabes en que planta ni por donde entrar

### 3. Empiezas a preguntar

A seguridad, a otros estudiantes o a cualquiera que pase

### 4. Pierdes tiempo

Llegas con menos confianza, te dan mal las indicaciones, etc.

**Tesis Inicial:** No solo se debe orientar en un mapa, hay que reducir la incertidumbre para encontrar bloques, aulas o laboratorios dentro de un campus nuevo

## EL PROBLEMA: Lo difícil no es ubicar un punto, sino llegar con certeza

### Para quien llega al campus

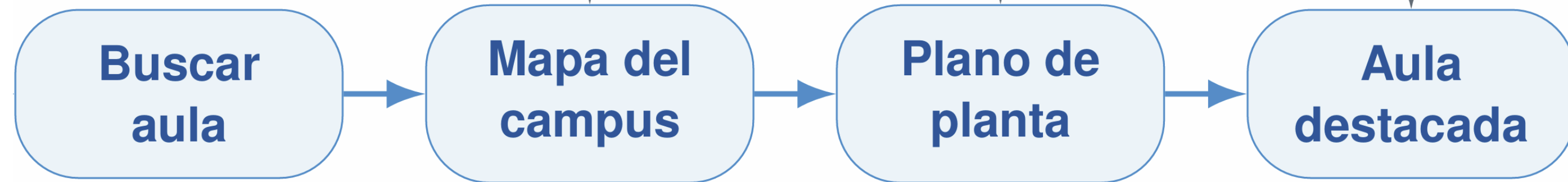
- No conoce bloques, accesos ni entradas.
- Necesita una ruta simple.
- Quiere pasar de campus a aula exacta.

### Para la Universidad

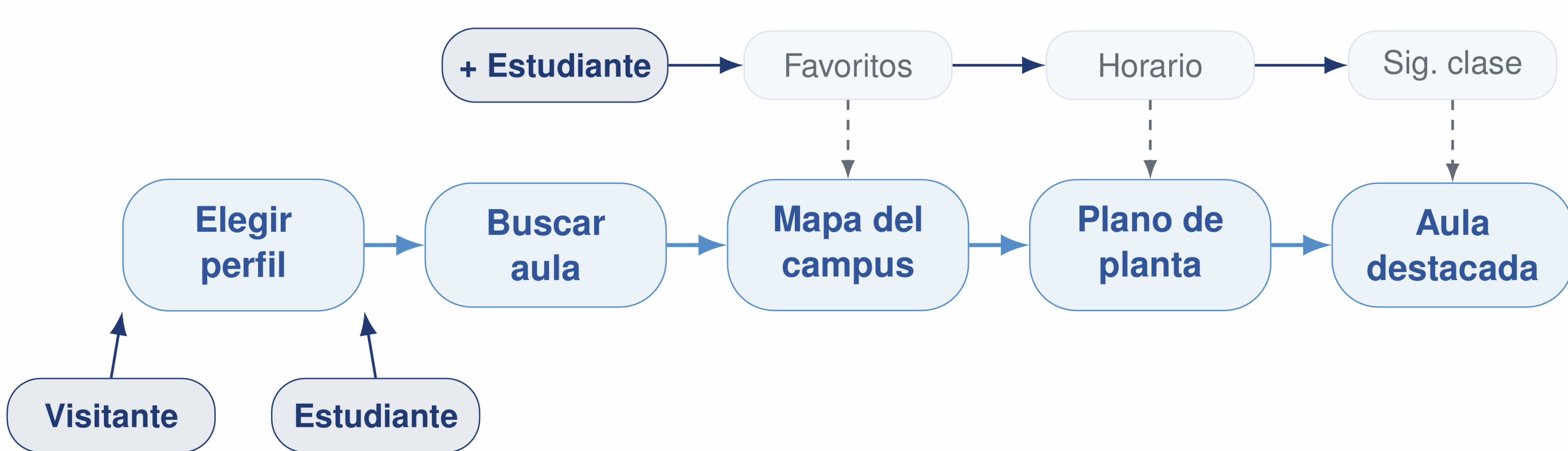
- Mejorar la experiencia de bienvenida y movilidad.
- Hacer visible la infraestructura del campus.
- Reducir indicaciones informales.

**Oportunidad:** una sola app que conecte ubicación, búsqueda, planos y contexto académico.

# LA PROPUESTA: Del nombre de una aula a una ruta entendible



**FLUJO BASE:** Dos perfiles, ruta al aula. Estudiante accede a horarios, tareas sobre el mismo mapa



# EL PRODUCTO QUE CONSTRUÍ CON FLUTTER

## Mapa interactivo

Bloques visibles y navegables.

## Planos por planta

Vista interior por edificio.

## Favoritos y Horario

Para Rutina del estudiante



## Búsqueda de aulas

Acceso directo por nombre

## Tareas y siguiente clase

Acompañamiento diario

## Geolocalización

Ubicación actual y seguimiento

## EXPERIENCIA: Misma base, dos recorridos distintos

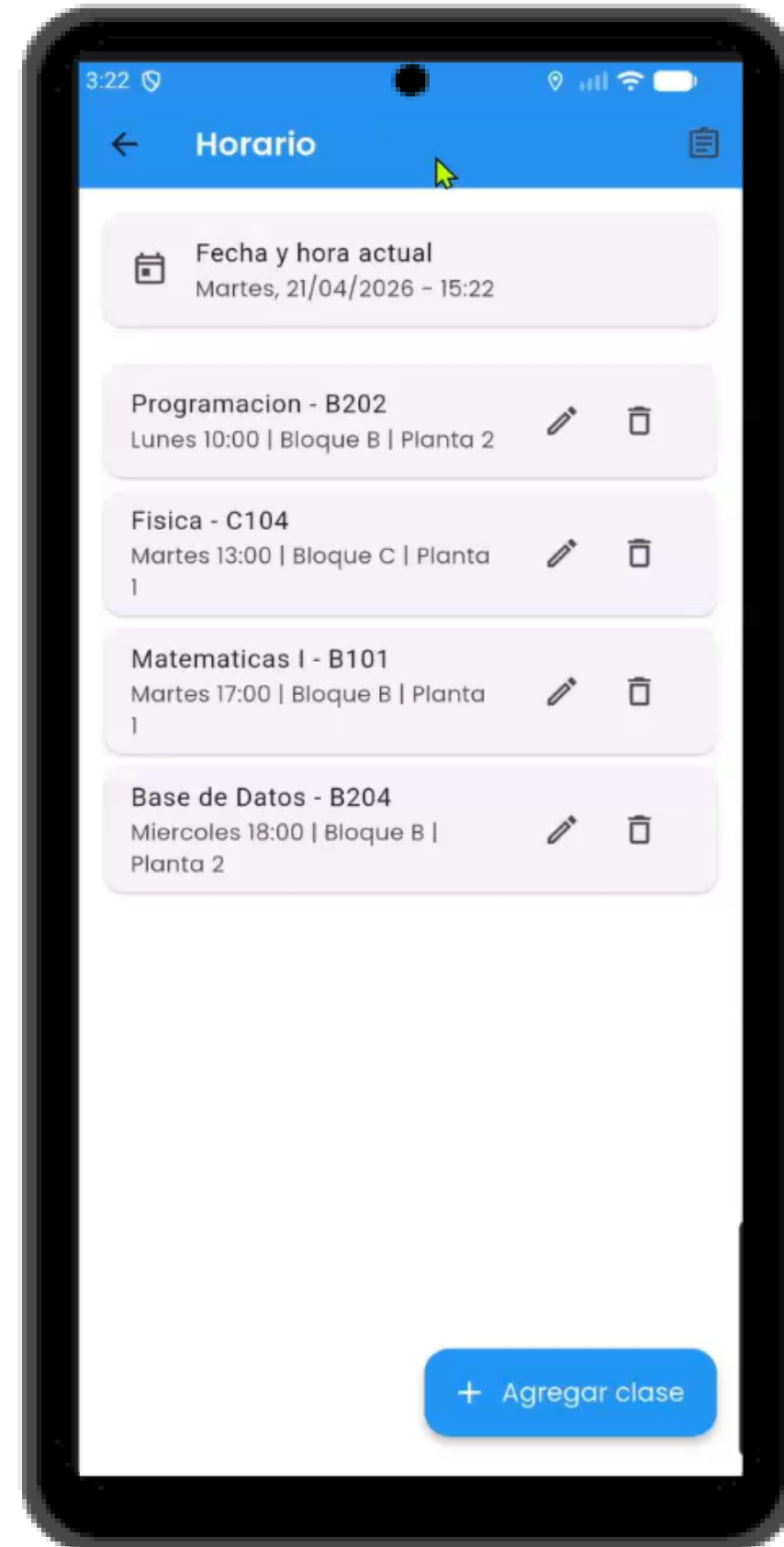
### Visitante

- Entra y busca sin fricción
- Se orienta por bloque y planta
- Recibe solo lo necesario para llegar

Simplicidad

Velocidad

Claridad



### Estudiante

- Guarda aulas frecuentes en favoritos
- Consulta su horario y la siguiente clase
- Usa tareas y navegación como flujo continuo

Rutina

Contexto

Persistencia

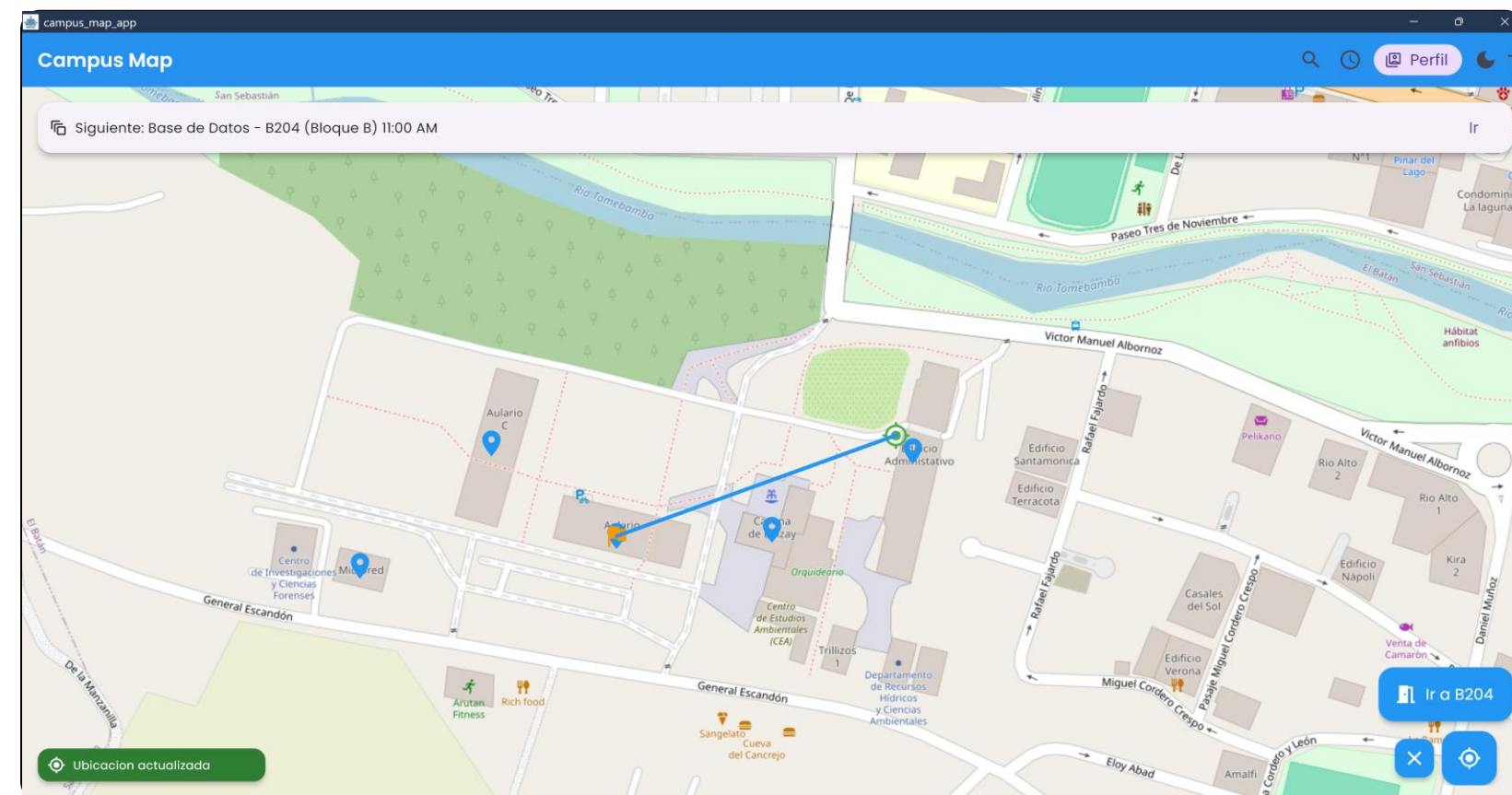


**Importante:** No hacer dos apps, sino adaptar la experiencia

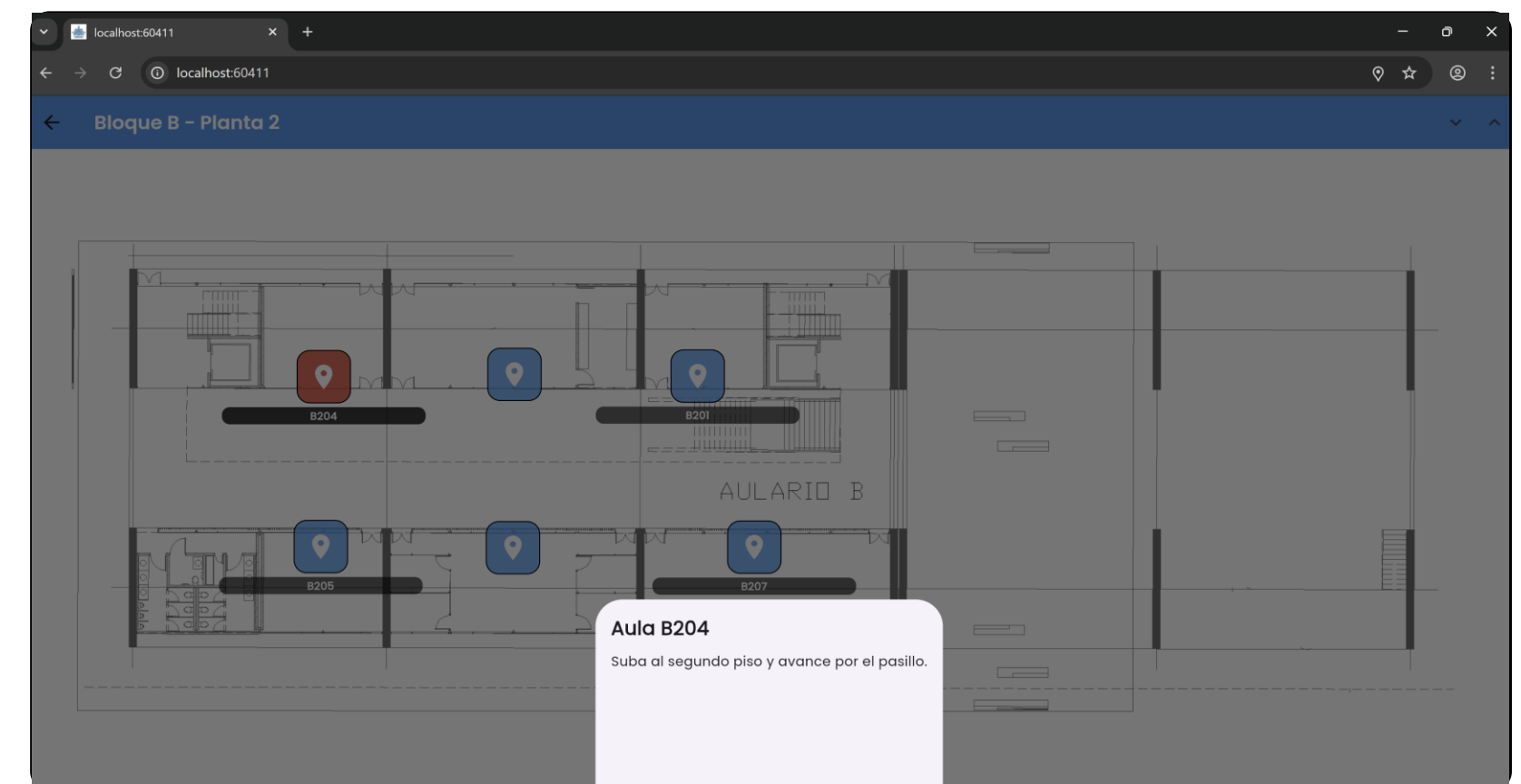
# ESCALA: Del campus completo al detalle de una planta

Lo importante aquí no es la imagen, sino el modelo: los espacios se alimentan desde datos editables, lo que permite ampliar edificios y aulas sin rehacer la interfaz.

Escala 1: campus como sistema de bloques y trayectos



Escala 2: abrir la planta correcta y resaltar el aula dentro del edificio



# DECISIONES: Lo que de verdad cambió la experiencia

## Priorizar la búsqueda

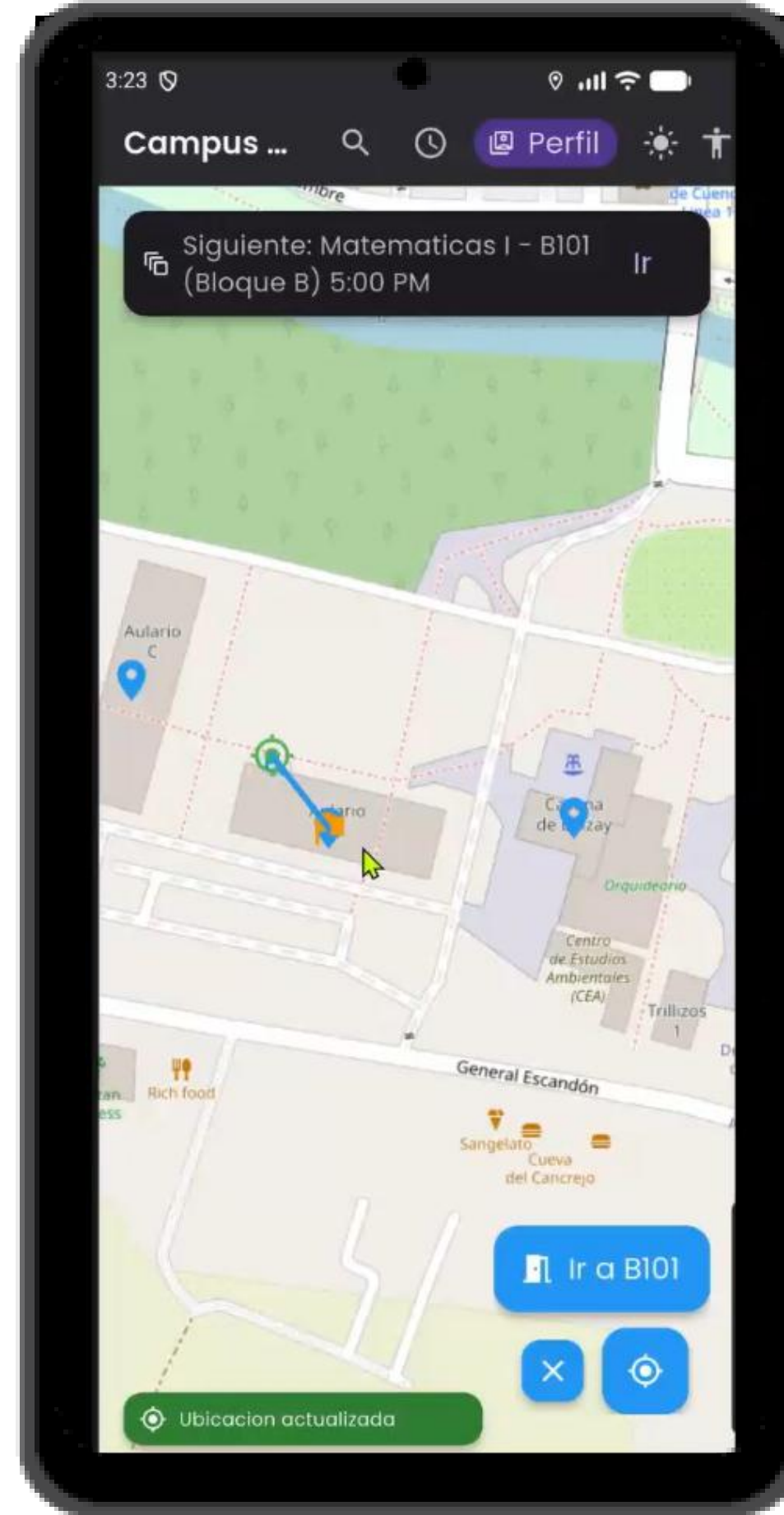
Evita exploración innecesaria

## Separa perfiles

Visitante necesita llegar,  
estudiante necesita a diario.

## Mantener datos editables

JSON y planos permiten  
crecer sin bloqueos



## Publicar en WEB

Para eventos se puede desplegar  
un link antes de instalar una app

## No saturar la primera pantalla

Orientación, luego capas de valor  
(Horario, tareas, etc...)

Idea: Problemas especiales deben  
ser auto explicativos para no hacer  
al usuario entender la app

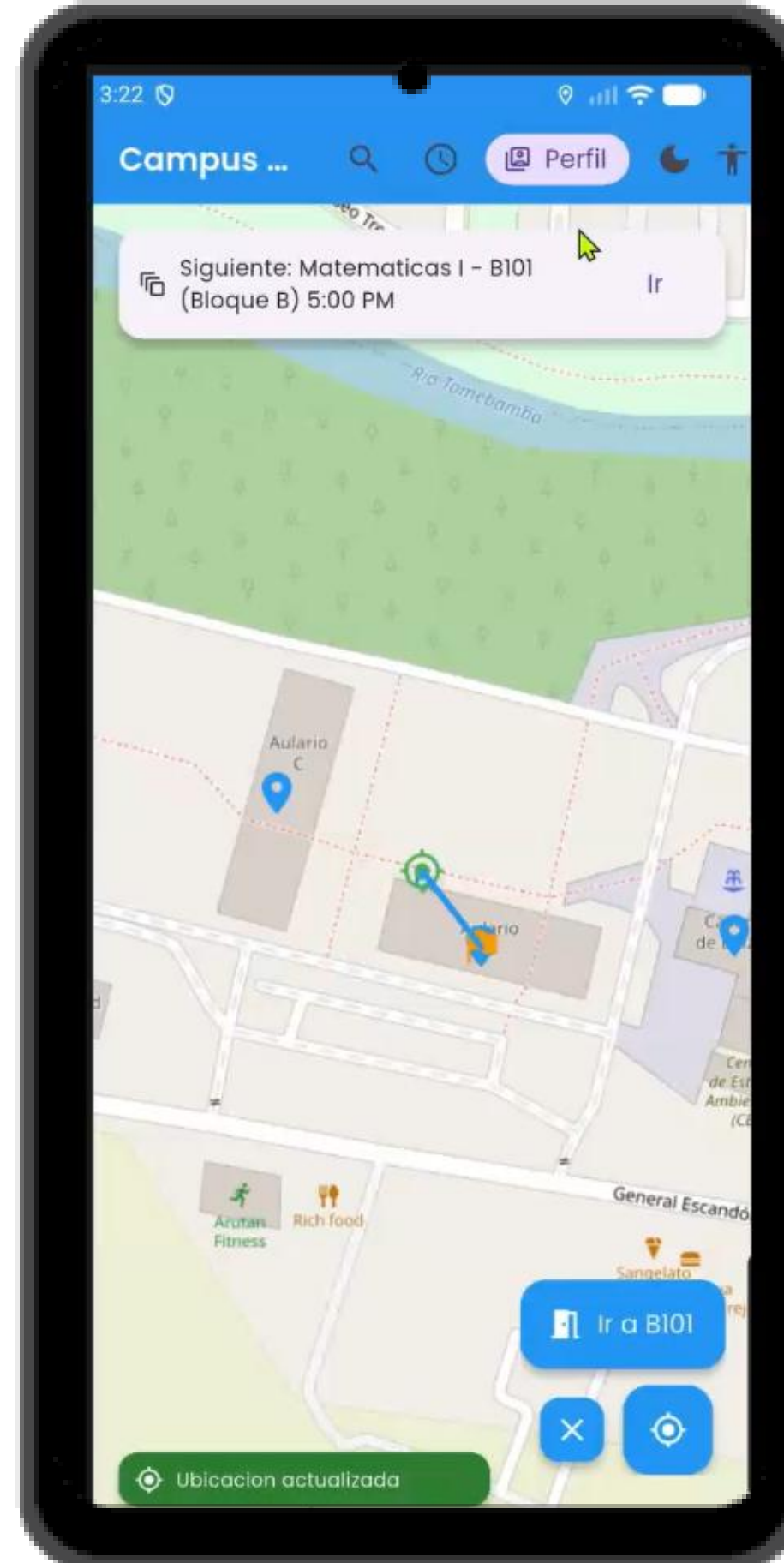
# POR QUE FLUTTER: Velocidad, Alcance y buenas prácticas

Una sola base: Mismo producto  
diferentes plataformas

Ecosistema suficiente: Mapas,  
geolocalización, UI, listos integrados

Consistencia: Misma experiencia  
diferentes dispositivos

Despliegue útil: Web acceso  
inmediato, apps recurrencia



## Paquetes aplicados :

- **Provider:** tema y configuración visual
- **Material 3:** componentes modernos y diseño consistente
- **flutter\_map:** renderizado del mapa del campus
- **OpenStreetMap:** mapa base (teselas)
- **geolocator:** geolocalización y permisos de ubicación
- **shared\_preferences:** guardar rol, favoritos, horario y tareas
- **JSON (assets):** aulas, coordenadas y datos por planta
- **latlong2:** cálculo de distancias y coordenadas
- Una sola base de código: web, móvil y escritorio

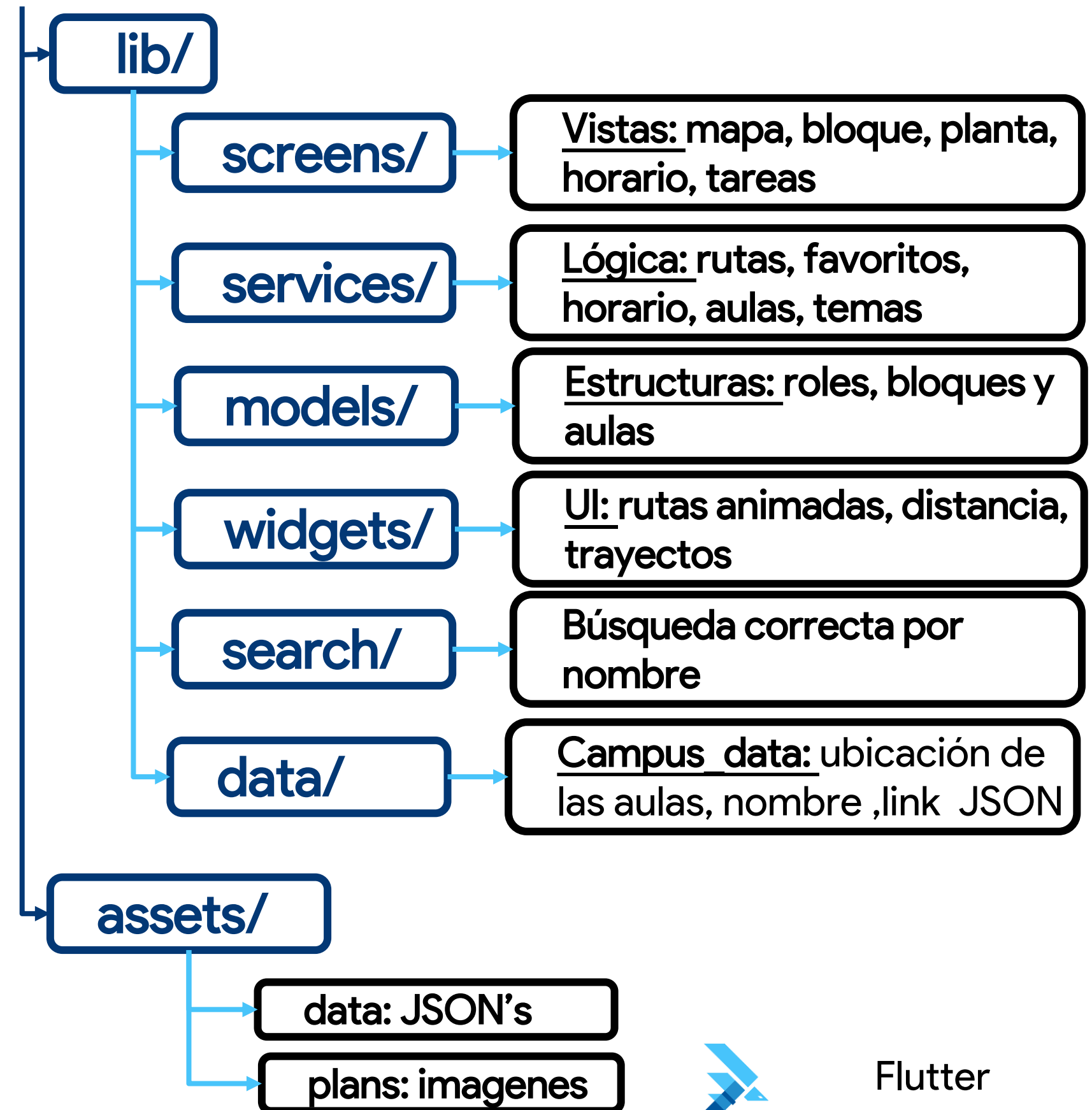
# BUENAS PRÁCTICAS: Reglas que seguimos para crecer sin romper

**Separar por responsabilidad:** Cada parte quedo en su lugar: screens para flujo, services para datos y logica, y widgets para piezas reutilizables.

**Widgets pequeños y claros:** Preferimos

**StatelessWidget** cuando es posible, usamos **const** y evitamos meter lógica de negocio dentro de build().

**Cierre con calidad:** Estados y errores visibles, nombres consistentes y cierre con analyze, test y format



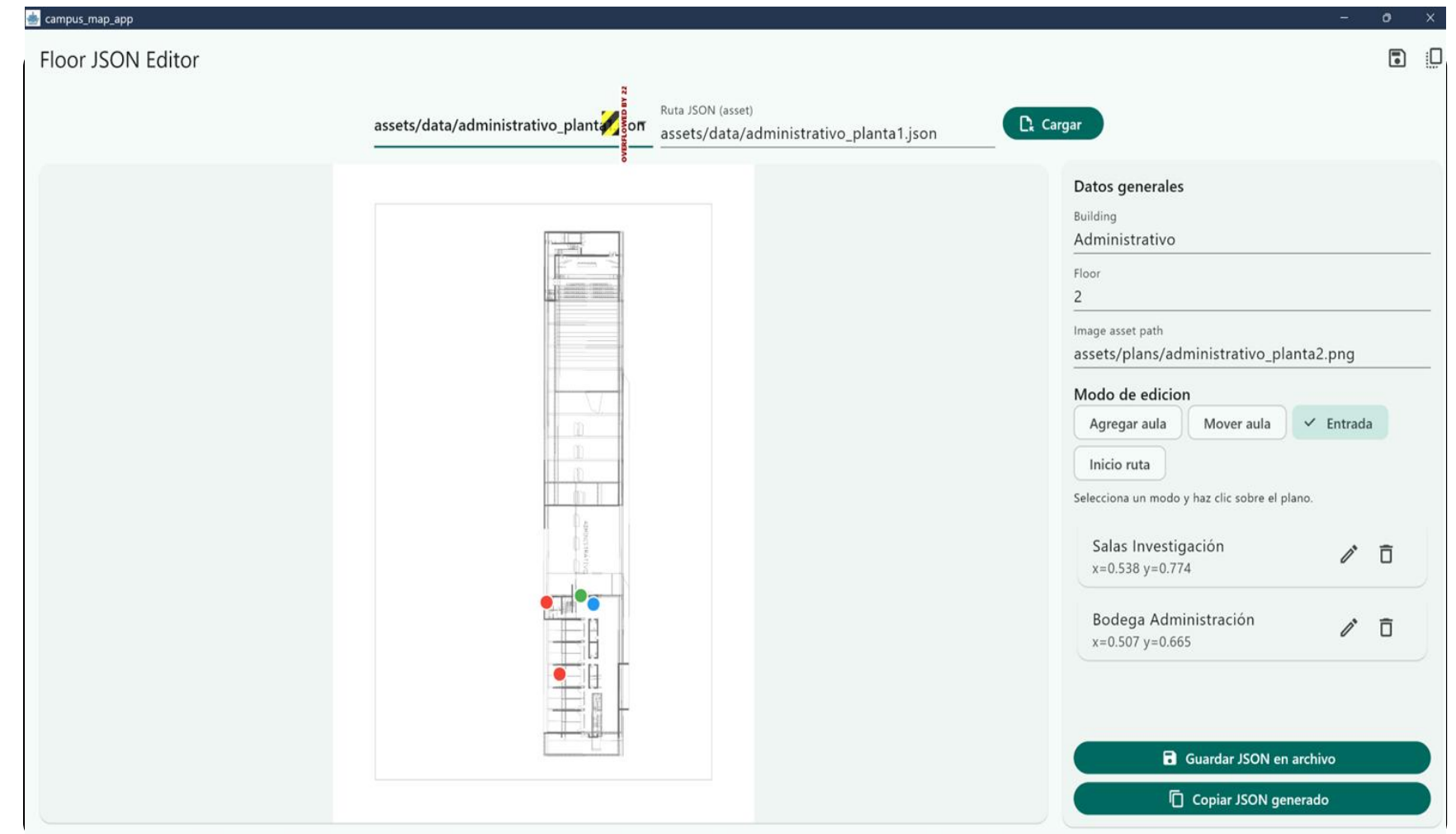
## APRENDIZAJES: Lo que me dejo construir esta app

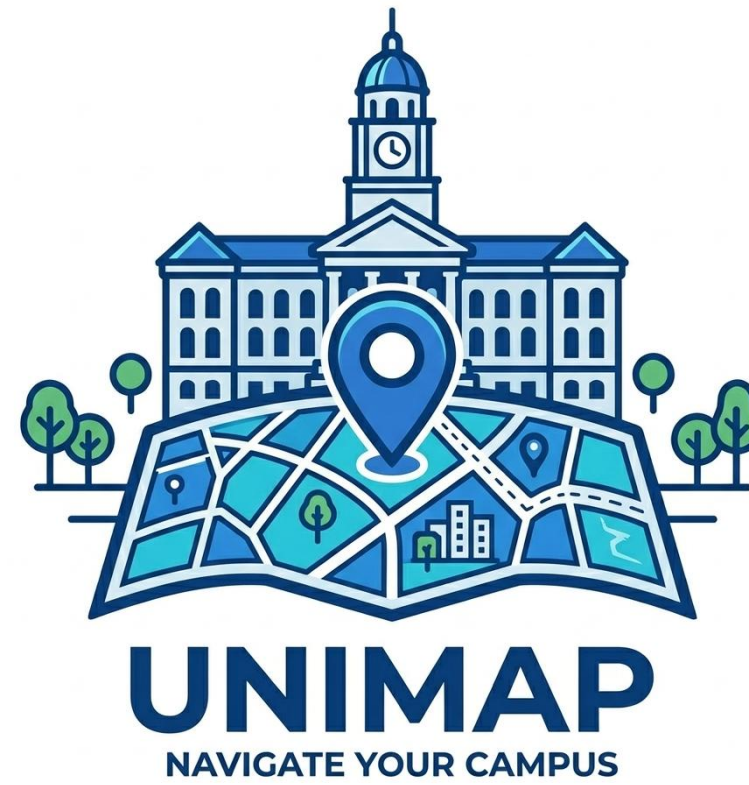
- Un problema pequeño y concreto suele producir un mejor punto inicial
- La capa de datos importa tanto como la interfaz
- En navegación, claridad vale más que sofisticación

**Valor:** Flutter puede ser una herramienta adecuada para resolver necesidades locales, universitarias y concretas

**Aprendizaje:** Pensar en la app como una experiencia de llegada y no solo como un visor

Herramienta: Editor gráfico de JSON





# GRACIAS

Flutter también puede resolver  
problemas muy cercanos, cotidianos o  
locales